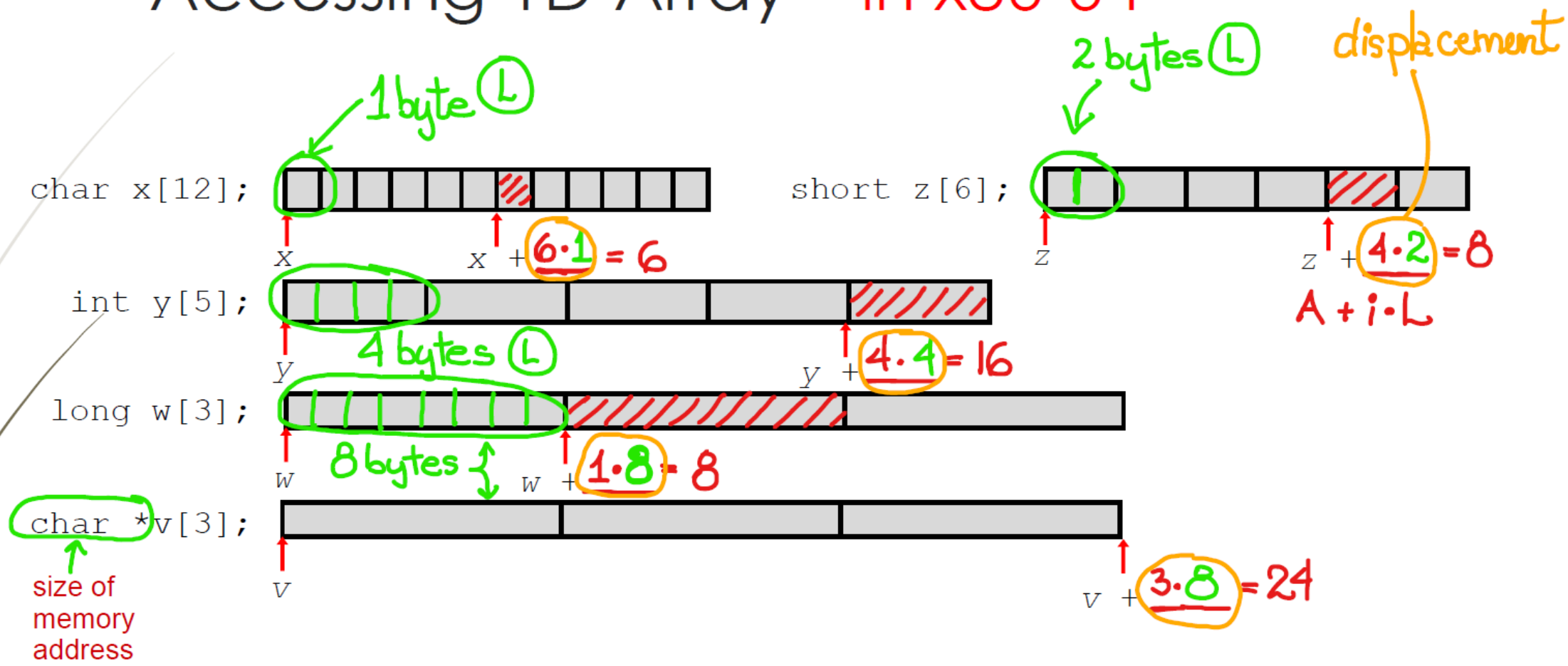
Accessing 1D Array – in x86-64

Arrays declared using C syntax



Manipulating 1D array – Example - `sums.s` - Part 1

- Register **`%rdi`** contains starting address of array (base address of array)
- Register **`%esi`** contains size of array (N) → in terms of cell #
- Register **`%ecx`** contains array index
- Register **`%al`** or **`%ax`** contains the running sum

```
    .globl   sumChar
sumChar:
    movl     $0,      %eax    sum = 0
    movl     $0,      %ecx    i = 0
loopChar:
    cmpl     %ecx,    %esi
    jle      endloop1
    addb     (%rdi,%rcx,1),   %al
    incl     %ecx
    jmp      loopChar
endloop1:
    ret
```

We loop while
N > i (**g**) is True
We jump to endloop1
when N > i is False,
i.e., when N <= I (**le**)

base
displacement
char

Compute memory address
of each element in array
and access its value

```
    .globl   sumShort
sumShort:
    xorl     %eax, %eax
    xorl     %ecx, %ecx
    jmp      cond1
loopShort:
    addw     (%rdi, %rcx, 2), %ax
    incl     %ecx
cond1:
    cmpl     %esi, %ecx
    jne      loopShort
    ret
```

short

Manipulating 1D array – Example - `sums.s` - Part 2

- Register **`%rdi`** contains starting address of array (base address of array)
- Register **`%esi`** contains size of array (N) → in terms of cell #
- Register **`%ecx`** contains array index
- Register **`%eax`** or **`%rax`** contains the running sum

```
    .globl  sumInt
sumInt:
    xorl    %eax, %eax
    xorl    %ecx, %ecx
    jmp     cond2
loopInt:
    addl    (%rdi, %rcx, 4), %eax
    incl    %ecx
cond2:
    cmpl    %esi, %ecx
    jne     loopInt
    ret
```

ints

Compute memory address of each element in array and access its value

```
    .globl  sumLong
sumLong:
    movq    $0,     %rax
    movl    $0,     %ecx
loopLong:
    cmpl    %ecx,   %esi
    jle     endloop
    addq    (%rdi,%rcx,8), %rax
    incl    %ecx
    jmp     loopLong
endloop:
    ret
```

longs